Introduction to MLIR

Session 1, Part 2 — GPU

SC26 Tutorial

Setup

- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.

Setup

- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.
- **Fresh runtime.** Nothing from Part 1 carries over, so the notebook downloads the toolchain and builds the plugin again.

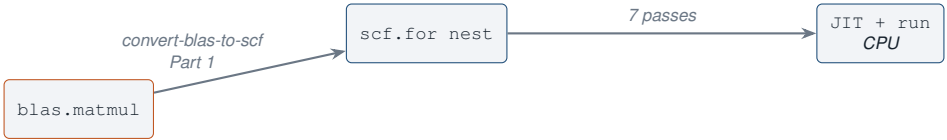
Setup

- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.
- **Fresh runtime.** Nothing from Part 1 carries over, so the notebook downloads the toolchain and builds the plugin again.
- **We start from the Part 1 solution.** The notebook clones the `session1-part1-solution` branch, where `blas.matmul` is already defined.

Setup

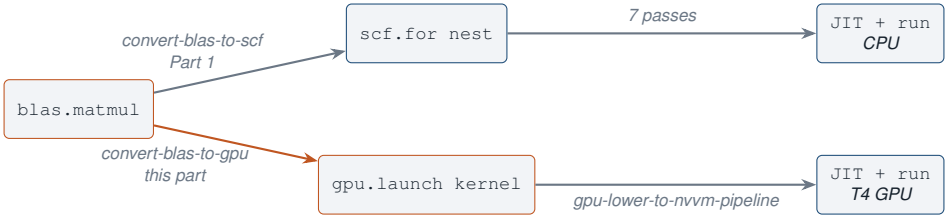
- **Switch the runtime to a GPU:** `Change runtime type` → `T4 GPU`, before running any cell.
- **Fresh runtime.** Nothing from Part 1 carries over, so the notebook downloads the toolchain and builds the plugin again.
- **We start from the Part 1 solution.** The notebook clones the `session1-part1-solution` branch, where `blas.matmul` is already defined.
- **One exercise:** GPU lowering.

GPU Lowering



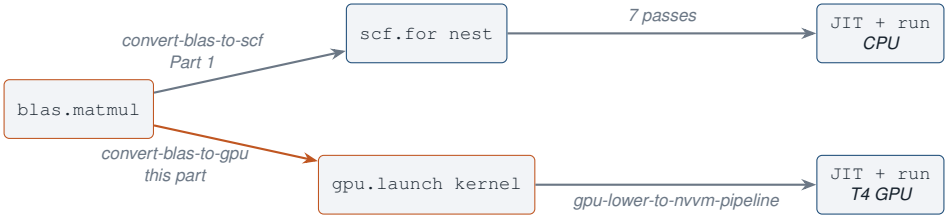
- In Part 1, we lowered `blas.matmul` to loops and ran it on the CPU.

GPU Lowering



- In Part 1, we lowered `blas.matmul` to loops and ran it on the CPU.
- In this part, we lower the same op to a `gpu.launch` kernel and run it on a GPU.

GPU Lowering

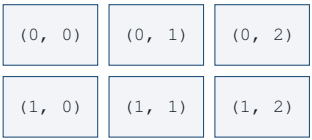


- In Part 1, we lowered `blas.matmul` to loops and ran it on the CPU.
- In this part, we lower the same op to a `gpu.launch` kernel and run it on a GPU.
- First we examine that kernel.

Step 1 — Loop-Level GPU Kernel

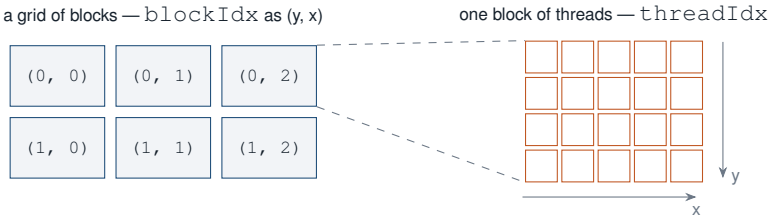
Blocks and Threads

a grid of blocks — `blockIdx` as (y, x)



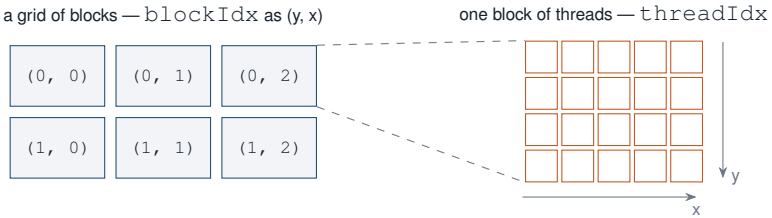
- A kernel runs the same body on many threads at once.

Blocks and Threads



- A kernel runs the same body on many threads at once.
- Threads are grouped into blocks, and blocks into a grid. Each can be up to 3D: x, y and z.

Blocks and Threads



- A kernel runs the same body on many threads at once.
- Threads are grouped into blocks, and blocks into a grid. Each can be up to 3D: x, y and z.
- Every thread knows its `blockIdx`, its `threadIdx` and the block size, `blockDim`. Its global position in each dimension is `blockIdx * blockDim + threadIdx`.

gpu.launch

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
  // the body: run once by every thread
  gpu.terminator
}
```

▶ Colab 2.10

gpu.launch

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
  // the body: run once by every thread
  gpu.terminator
}
```

- `blocks(...) in (...)`: how many blocks, in x, y and z. Here, one.

gpu.launch

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
  // the body: run once by every thread
  gpu.terminator
}
```

- `blocks(...) in (...)`: how many blocks, in x, y and z. Here, one.
- `threads(...) in (...)`: how many threads in each block. Here $5 \times 4 = 20$. Inside the body `%blockX, %blockY, %blockZ` name those sizes: `blockDim`.

gpu.launch

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
  // the body: run once by every thread
  gpu.terminator
}
```

- `blocks(...) in (...)`: how many blocks, in x, y and z. Here, one.
- `threads(...) in (...)`: how many threads in each block. Here $5 \times 4 = 20$. Inside the body `%blockX, %blockY, %blockZ` name those sizes: `blockDim`.
- `%bx, %by, %bz` and `%tx, %ty, %tz` are bound inside the body: the block and thread ids, CUDA's `blockIdx` and `threadIdx`.

gpu.launch

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
  // the body: run once by every thread
  gpu.terminator
}
```

- `blocks(...) in (...)`: how many blocks, in x, y and z. Here, one.
- `threads(...) in (...)`: how many threads in each block. Here $5 \times 4 = 20$. Inside the body `%blockX, %blockY, %blockZ` name those sizes: `blockDim`.
- `%bx, %by, %bz` and `%tx, %ty, %tz` are bound inside the body: the block and thread ids, CUDA's `blockIdx` and `threadIdx`.
- Every thread runs the body, and `gpu.terminator` ends it.

`matmul_gpu.mlir`, Step 1 of 2 — Launch

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %a_u = memref.cast %a : memref<4x3xf32> to memref<*xf32>
  %b_u = memref.cast %b : memref<3x5xf32> to memref<*xf32>
  %out_u = memref.cast %out : memref<4x5xf32> to memref<*xf32>
  gpu.host_register %a_u : memref<*xf32>
  gpu.host_register %b_u : memref<*xf32>
  gpu.host_register %out_u : memref<*xf32>
  %c1 = arith.constant 1 : index
  %c3 = arith.constant 3 : index
  %c4 = arith.constant 4 : index
  %c5 = arith.constant 5 : index
  gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
             threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
    // the body: next slides
    gpu.terminator
  }
  return
}
```

`matmul_gpu.mlir`, Step 1 of 2 — Launch

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %a_u = memref.cast %a : memref<4x3xf32> to memref<*xf32>
  %b_u = memref.cast %b : memref<3x5xf32> to memref<*xf32>
  %out_u = memref.cast %out : memref<4x5xf32> to memref<*xf32>
  gpu.host_register %a_u : memref<*xf32>
  gpu.host_register %b_u : memref<*xf32>
  gpu.host_register %out_u : memref<*xf32>
  %c1 = arith.constant 1 : index
  %c3 = arith.constant 3 : index
  %c4 = arith.constant 4 : index
  %c5 = arith.constant 5 : index
  gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
             threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
    // the body: next slides
    gpu.terminator
  }
  return
}
```

- The `memref.cast` + `gpu.host_register` lines make the host buffers reachable from the GPU.

`matmul_gpu.mlir`, Step 1 of 2 — Launch

```
func.func @matmul(%a: memref<4x3xf32>, %b: memref<3x5xf32>,
                  %out: memref<4x5xf32>) {
  %a_u = memref.cast %a : memref<4x3xf32> to memref<*xf32>
  %b_u = memref.cast %b : memref<3x5xf32> to memref<*xf32>
  %out_u = memref.cast %out : memref<4x5xf32> to memref<*xf32>
  gpu.host_register %a_u : memref<*xf32>
  gpu.host_register %b_u : memref<*xf32>
  gpu.host_register %out_u : memref<*xf32>
  %c1 = arith.constant 1 : index
  %c3 = arith.constant 3 : index
  %c4 = arith.constant 4 : index
  %c5 = arith.constant 5 : index
  gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
             threads(%tx, %ty, %tz) in (%blockX = %c5, %blockY = %c4, %blockZ = %c1) {
    // the body: next slides
    gpu.terminator
  }
  return
}
```

- The `memref.cast` + `gpu.host_register` lines make the host buffers reachable from the GPU.
- One block of 5×4 threads: one thread per element of the 4×5 `out`.

Thread to Element Mapping

	tx = 0	tx = 1	tx = 2	tx = 3	tx = 4
ty = 0	(0, 0)	(0, 1)	(0, 2)	(0, 3)	(0, 4)
ty = 1	(1, 0)	(1, 1)	(1, 2)	(1, 3)	(1, 4)
ty = 2	(2, 0)	(2, 1)	(2, 2)	(2, 3)	(2, 4)
ty = 3	(3, 0)	(3, 1)	(3, 2)	(3, 3)	(3, 4)

`out`, 4×5: each cell is the thread `(ty, tx)` that computes it

Thread to Element Mapping

	tx = 0	tx = 1	tx = 2	tx = 3	tx = 4
ty = 0	(0, 0)	(0, 1)	(0, 2)	(0, 3)	(0, 4)
ty = 1	(1, 0)	(1, 1)	(1, 2)	(1, 3)	(1, 4)
ty = 2	(2, 0)	(2, 1)	(2, 2)	(2, 3)	(2, 4)
ty = 3	(3, 0)	(3, 1)	(3, 2)	(3, 3)	(3, 4)

`out`, 4×5: each cell is the thread `(ty, tx)` that computes it

- y spans the 4 rows of `out`, x its 5 columns. Each thread computes one element of the output.

matmul_gpu.mlir, Step 2 of 2 — Body

```
gpu.launch blocks(%bx, %by, %bz) ... threads(%tx, %ty, %tz) ... {
  %rowBase = arith.muli %by, %blockY : index
  %row = arith.addi %rowBase, %ty : index
  %colBase = arith.muli %bx, %blockX : index
  %col = arith.addi %colBase, %tx : index
  %c0 = arith.constant 0 : index
  %step = arith.constant 1 : index
  %init = memref.load %out[%row, %col] : memref<4x5xf32>
  %acc = scf.for %k = %c0 to %c3 step %step iter_args(%s = %init) -> (f32) {
    %av = memref.load %a[%row, %k] : memref<4x3xf32>
    %bv = memref.load %b[%k, %col] : memref<3x5xf32>
    %prod = arith.mulf %av, %bv : f32
    %s2 = arith.addf %s, %prod : f32
    scf.yield %s2 : f32
  }
  memref.store %acc, %out[%row, %col] : memref<4x5xf32>
  gpu.terminator
}
```

matmul_gpu.mlir, Step 2 of 2 — Body

```
gpu.launch blocks(%bx, %by, %bz) ... threads(%tx, %ty, %tz) ... {
  %rowBase = arith.muli %by, %blockY : index
  %row = arith.addi %rowBase, %ty : index
  %colBase = arith.muli %bx, %blockX : index
  %col = arith.addi %colBase, %tx : index
  %c0 = arith.constant 0 : index
  %step = arith.constant 1 : index
  %init = memref.load %out[%row, %col] : memref<4x5xf32>
  %acc = scf.for %k = %c0 to %c3 step %step iter_args(%s = %init) -> (f32) {
    %av = memref.load %a[%row, %k] : memref<4x3xf32>
    %bv = memref.load %b[%k, %col] : memref<3x5xf32>
    %prod = arith.mulf %av, %bv : f32
    %s2 = arith.addf %s, %prod : f32
    scf.yield %s2 : f32
  }
  memref.store %acc, %out[%row, %col] : memref<4x5xf32>
  gpu.terminator
}
```

- Global row and column: block id $\times$ block size + thread id.

matmul_gpu.mlir, Step 2 of 2 — Body

```
gpu.launch blocks(%bx, %by, %bz) ... threads(%tx, %ty, %tz) ... {
  %rowBase = arith.muli %by, %blockY : index
  %row = arith.addi %rowBase, %ty : index
  %colBase = arith.muli %bx, %blockX : index
  %col = arith.addi %colBase, %tx : index
  %c0 = arith.constant 0 : index
  %step = arith.constant 1 : index
  %init = memref.load %out[%row, %col] : memref<4x5xf32>
  %acc = scf.for %k = %c0 to %c3 step %step iter_args(%s = %init) -> (f32) {
    %av = memref.load %a[%row, %k] : memref<4x3xf32>
    %bv = memref.load %b[%k, %col] : memref<3x5xf32>
    %prod = arith.mulf %av, %bv : f32
    %s2 = arith.addf %s, %prod : f32
    scf.yield %s2 : f32
  }
  memref.store %acc, %out[%row, %col] : memref<4x5xf32>
  gpu.terminator
}
```

- Global row and column: block id $\times$ block size + thread id.
- Thread `(%ty, %tx)` reads, accumulates and writes `out[%row, %col]`.

matmul_gpu.mlir, Step 2 of 2 — Body

```
gpu.launch blocks(%bx, %by, %bz) ... threads(%tx, %ty, %tz) ... {
  %rowBase = arith.muli %by, %blockY : index
  %row = arith.addi %rowBase, %ty : index
  %colBase = arith.muli %bx, %blockX : index
  %col = arith.addi %colBase, %tx : index
  %c0 = arith.constant 0 : index
  %step = arith.constant 1 : index
  %init = memref.load %out[%row, %col] : memref<4x5xf32>
  %acc = scf.for %k = %c0 to %c3 step %step iter_args(%s = %init) -> (f32) {
    %av = memref.load %a[%row, %k] : memref<4x3xf32>
    %bv = memref.load %b[%k, %col] : memref<3x5xf32>
    %prod = arith.mulf %av, %bv : f32
    %s2 = arith.addf %s, %prod : f32
    scf.yield %s2 : f32
  }
  memref.store %acc, %out[%row, %col] : memref<4x5xf32>
  gpu.terminator
}
```

- Global row and column: block id $\times$ block size + thread id.
- Thread `(%ty, %tx)` reads, accumulates and writes `out[%row, %col]`.
- The reduction over `%k` is Part 1's loop, on row `%row` of `a` and column `%col` of `b`.

`matmul_gpu.mlir`, Step 2 of 2 — Body

```
gpu.launch blocks(%bx, %by, %bz) ... threads(%tx, %ty, %tz) ... {
  %rowBase = arith.muli %by, %blockY : index
  %row = arith.addi %rowBase, %ty : index
  %colBase = arith.muli %bx, %blockX : index
  %col = arith.addi %colBase, %tx : index
  %c0 = arith.constant 0 : index
  %step = arith.constant 1 : index
  %init = memref.load %out[%row, %col] : memref<4x5xf32>
  %acc = scf.for %k = %c0 to %c3 step %step iter_args(%s = %init) -> (f32) {
    %av = memref.load %a[%row, %k] : memref<4x3xf32>
    %bv = memref.load %b[%k, %col] : memref<3x5xf32>
    %prod = arith.mulf %av, %bv : f32
    %s2 = arith.addf %s, %prod : f32
    scf.yield %s2 : f32
  }
  memref.store %acc, %out[%row, %col] : memref<4x5xf32>
  gpu.terminator
}
```

- Global row and column: block id $\times$ block size + thread id.
- Thread `(%ty, %tx)` reads, accumulates and writes `out[%row, %col]`.
- The reduction over `%k` is Part 1's loop, on row `%row` of `a` and column `%col` of `b`.
- No explicit loops in the code over rows and columns: the 2D grid of threads spans those loops.

▶ Colab 2.14, 2.16

Lowering and Execution

```
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin
                           cubin-features=+ptx78}
```

Lowering and Execution

```
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin
                           cubin-features=+ptx78}
```

- Moves the launch body out into a separate kernel, lowers it to NVVM, and compiles it to a binary for this GPU.

Lowering and Execution

```
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin
                           cubin-features=+ptx78}
```

- Moves the launch body out into a separate kernel, lowers it to NVVM, and compiles it to a binary for this GPU.
- Rewrites the host side to launch that kernel through the CUDA driver.

Lowering and Execution

```
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin
                           cubin-features=+ptx78}
```

- Moves the launch body out into a separate kernel, lowers it to NVVM, and compiles it to a binary for this GPU.
- Rewrites the host side to launch that kernel through the CUDA driver.
- The JIT needs the CUDA runtime wrapper library in `shared_libs`, and `engine.initialize()` to load the compiled kernel.

Lowering and Execution

```
gpu-lower-to-nvvm-pipeline{cubin-chip=sm_75 cubin-format=bin
                           cubin-features=+ptx78}
```

- Moves the launch body out into a separate kernel, lowers it to NVVM, and compiles it to a binary for this GPU.
- Rewrites the host side to launch that kernel through the CUDA driver.
- The JIT needs the CUDA runtime wrapper library in `shared_libs`, and `engine.initialize()` to load the compiled kernel.

```
OK
```

Step 2 — Lowering `blas.matmul` to GPU

▶ Colab 2.22, 2.23, 2.25

Reference: `MatVecToGPUPattern`, Step 1 of 5 — Setup

the C++

```
struct MatVecToGPUPattern : public OpRewritePattern<MatVecOp> {
  using OpRewritePattern::OpRewritePattern;
  LogicalResult
  matchAndRewrite(MatVecOp op,
                  PatternRewriter &rewriter) const override {
    Location loc = op.getLoc();
    Value mat = op.getMat();
    Value vec = op.getVec();
    Value out = op.getOut();
    // memref.cast + gpu::HostRegisterOp for mat, vec, out
    Value c0 = arith::ConstantIndexOp::create(rewriter, loc, 0);
    Value c1 = arith::ConstantIndexOp::create(rewriter, loc, 1);
    Value numRows = memref::DimOp::create(rewriter, loc, mat, c0);
    Value numCols = memref::DimOp::create(rewriter, loc, mat, c1);
    // steps 2-4 go here
    rewriter.eraseOp(op);
    return success();
  }
};
```

the IR it generates

```
// memref.cast + gpu.host_register
%c0 = arith.constant 0 : index
%c1 = arith.constant 1 : index
%numRows = memref.dim %mat, %c0
%numCols = memref.dim %mat, %c1
```

▶ Colab 2.27

Reference: `MatVecToGPUPattern`, Step 1 of 5 — Setup

the C++

```
struct MatVecToGPUPattern : public OpRewritePattern<MatVecOp> {
  using OpRewritePattern::OpRewritePattern;
  LogicalResult
  matchAndRewrite(MatVecOp op,
                  PatternRewriter &rewriter) const override {
    Location loc = op.getLoc();
    Value mat = op.getMat();
    Value vec = op.getVec();
    Value out = op.getOut();
    // memref.cast + gpu::HostRegisterOp for mat, vec, out
    Value c0 = arith::ConstantIndexOp::create(rewriter, loc, 0);
    Value c1 = arith::ConstantIndexOp::create(rewriter, loc, 1);
    Value numRows = memref::DimOp::create(rewriter, loc, mat, c0);
    Value numCols = memref::DimOp::create(rewriter, loc, mat, c1);
    // steps 2-4 go here
    rewriter.eraseOp(op);
    return success();
  }
};
```

the IR it generates

```
// memref.cast + gpu.host_register
%c0 = arith.constant 0 : index
%c1 = arith.constant 1 : index
%numRows = memref.dim %mat, %c0
%numCols = memref.dim %mat, %c1
```

- `MatVecToSCFPattern`'s start, plus step 1's host registration (six lines, folded here).

Reference: `MatVecToGPUPattern`, Step 1 of 5 — Setup

the C++

```
struct MatVecToGPUPattern : public OpRewritePattern<MatVecOp> {
  using OpRewritePattern::OpRewritePattern;
  LogicalResult
  matchAndRewrite(MatVecOp op,
                  PatternRewriter &rewriter) const override {
    Location loc = op.getLoc();
    Value mat = op.getMat();
    Value vec = op.getVec();
    Value out = op.getOut();
    // memref.cast + gpu::HostRegisterOp for mat, vec, out
    Value c0 = arith::ConstantIndexOp::create(rewriter, loc, 0);
    Value c1 = arith::ConstantIndexOp::create(rewriter, loc, 1);
    Value numRows = memref::DimOp::create(rewriter, loc, mat, c0);
    Value numCols = memref::DimOp::create(rewriter, loc, mat, c1);
    // steps 2-4 go here
    rewriter.eraseOp(op);
    return success();
  }
};
```

the IR it generates

```
// memref.cast + gpu.host_register
%c0 = arith.constant 0 : index
%c1 = arith.constant 1 : index
%numRows = memref.dim %mat, %c0
%numCols = memref.dim %mat, %c1
```

- `MatVecToSCFPattern`'s start, plus step 1's host registration (six lines, folded here).
- In `MatMulToGPUPattern` all of this is provided. The exercise starts below it.

▶ Colab 2.27

Reference: `MatVecToGPUPattern`, Step 2 of 5 — Launch

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, numRows, c1, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  Value bx = launchOp.getBlockIds().x;
  Value blockDimX = launchOp.getBlockSize().x;
  Value tx = launchOp.getThreadIds().x;
  Value rowBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, tx);
  // steps 3 and 4 go here
}
```

the IR it generates

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %numRows, %blockY = %c1, %blockZ = %c1) {
  %rowBase = arith.muli %bx, %blockX : index
  %row = arith.addi %rowBase, %tx : index
  // steps 3 and 4 go here
}
```

Reference: `MatVecToGPUPattern`, Step 2 of 5 — Launch

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, numRows, c1, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  Value bx = launchOp.getBlockIds().x;
  Value blockDimX = launchOp.getBlockSize().x;
  Value tx = launchOp.getThreadIds().x;
  Value rowBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, tx);
  // steps 3 and 4 go here
}
```

the IR it generates

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %numRows, %blockY = %c1, %blockZ = %c1) {
  %rowBase = arith.muli %bx, %blockX : index
  %row = arith.addi %rowBase, %tx : index
  // steps 3 and 4 go here
}
```

- Grid size: `(1, 1, 1)`, block size: `(numRows, 1, 1)`.

Reference: `MatVecToGPUPattern`, Step 2 of 5 — Launch

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, numRows, c1, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  Value bx = launchOp.getBlockIds().x;
  Value blockDimX = launchOp.getBlockSize().x;
  Value tx = launchOp.getThreadIds().x;
  Value rowBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, tx);
  // steps 3 and 4 go here
}
```

the IR it generates

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %numRows, %blockY = %c1, %blockZ = %c1) {
  %rowBase = arith.muli %bx, %blockX : index
  %row = arith.addi %rowBase, %tx : index
  // steps 3 and 4 go here
}
```

- Grid size: `(1, 1, 1)`, block size: `(numRows, 1, 1)`.
- The body is a region: move the insertion point into it, as with Part 1's loops.

Reference: `MatVecToGPUPattern`, Step 2 of 5 — Launch

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, numRows, c1, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  Value bx = launchOp.getBlockIds().x;
  Value blockDimX = launchOp.getBlockSize().x;
  Value tx = launchOp.getThreadIds().x;
  Value rowBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, tx);
  // steps 3 and 4 go here
}
```

the IR it generates

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %numRows, %blockY = %c1, %blockZ = %c1) {
  %rowBase = arith.muli %bx, %blockX : index
  %row = arith.addi %rowBase, %tx : index
  // steps 3 and 4 go here
}
```

- Grid size: `(1, 1, 1)`, block size: `(numRows, 1, 1)`.
- The body is a region: move the insertion point into it, as with Part 1's loops.
- Then the block id, block size and thread id: `bx`, `blockDimX`, `tx`.

Reference: `MatVecToGPUPattern`, Step 2 of 5 — Launch

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, numRows, c1, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  Value bx = launchOp.getBlockIds().x;
  Value blockDimX = launchOp.getBlockSize().x;
  Value tx = launchOp.getThreadIds().x;
  Value rowBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, tx);
  // steps 3 and 4 go here
}
```

the IR it generates

```
gpu.launch blocks(%bx, %by, %bz) in (%gridX = %c1, %gridY = %c1, %gridZ = %c1)
           threads(%tx, %ty, %tz) in (%blockX = %numRows, %blockY = %c1, %blockZ = %c1) {
  %rowBase = arith.muli %bx, %blockX : index
  %row = arith.addi %rowBase, %tx : index
  // steps 3 and 4 go here
}
```

- Grid size: `(1, 1, 1)`, block size: `(numRows, 1, 1)`.
- The body is a region: move the insertion point into it, as with Part 1's loops.
- Then the block id, block size and thread id: `bx`, `blockDimX`, `tx`.
- From them the global row, `bx * blockDim.x + tx`.

Reference: `MatVecToGPUPattern`, Step 3 of 5 — Body

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{row});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  // the guard, the insertion point, j and acc: as in Part 1
  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{row, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  // prod, sum and the yield: as in Part 1
}
```

the IR it generates

```
%init = memref.load %out[%row]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {

  ...
  %m = memref.load %mat[%row, %j]

  %v = memref.load %vec[%j]

  ...
}
```

Reference: `MatVecToGPUPattern`, Step 3 of 5 — Body

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{row});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  // the guard, the insertion point, j and acc: as in Part 1
  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{row, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  // prod, sum and the yield: as in Part 1
}
```

the IR it generates

```
%init = memref.load %out[%row]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {

  ...
  %m = memref.load %mat[%row, %j]

  %v = memref.load %vec[%j]

  ...
}
```

- This is `MatVecToSCFPattern`'s body from Part 1, with the row index replaced: `row` where Part 1 had `i`.

Reference: `MatVecToGPUPattern`, Step 3 of 5 — Body

the C++

```
Value init = memref::LoadOp::create(rewriter, loc, out,
                                    ValueRange{row});
auto redLoop = scf::ForOp::create(rewriter, loc, c0, numCols, c1,
                                  ValueRange{init});
{
  // the guard, the insertion point, j and acc: as in Part 1
  Value matElem = memref::LoadOp::create(rewriter, loc, mat,
                                         ValueRange{row, j});
  Value vecElem = memref::LoadOp::create(rewriter, loc, vec,
                                         ValueRange{j});
  // prod, sum and the yield: as in Part 1
}
```

the IR it generates

```
%init = memref.load %out[%row]

%acc = scf.for %j = %c0 to %numCols step %c1
    iter_args(%a = %init) -> (f32) {

  ...
  %m = memref.load %mat[%row, %j]

  %v = memref.load %vec[%j]

  ...
}
```

- This is `MatVecToSCFPattern`'s body from Part 1, with the row index replaced: `row` where Part 1 had `i`.
- There is no row loop: the threads span the instances of the row loop.

Reference: `MatVecToGPUPattern`, Step 4 of 5 — Finish the Body

the C++

```
LogicalResult matchAndRewrite(MatVecOp op, ...) const override {
  // step 1: operands, host registration, c0, c1, numRows, numCols
  auto launchOp = gpu::LaunchOp::create(..., numRows, c1, c1);
  {
    // step 2: the insertion point, row = bx * blockDim.x + tx
    Value init = memref::LoadOp::create(..., out, ValueRange{row});
    auto redLoop = scf::ForOp::create(..., ValueRange{init});
    {
      // step 3: the reduction body
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{row});
    gpu::TerminatorOp::create(rewriter, loc);
  }
  rewriter.eraseOp(op);
  return success();
}
```

the IR it generates

```
gpu.launch ... {


  %init = memref.load %out[%row]
  %acc = scf.for %j ... {

    ...
  }
  memref.store %acc, %out[%row]

  gpu.terminator
}
```

Reference: `MatVecToGPUPattern`, Step 4 of 5 — Finish the Body

the C++

```
LogicalResult matchAndRewrite(MatVecOp op, ...) const override {
  // step 1: operands, host registration, c0, c1, numRows, numCols
  auto launchOp = gpu::LaunchOp::create(..., numRows, c1, c1);
  {
    // step 2: the insertion point, row = bx * blockDim.x + tx
    Value init = memref::LoadOp::create(..., out, ValueRange{row});
    auto redLoop = scf::ForOp::create(..., ValueRange{init});
    {
      // step 3: the reduction body
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{row});
    gpu::TerminatorOp::create(rewriter, loc);
  }
  rewriter.eraseOp(op);
  return success();
}
```

the IR it generates

```
gpu.launch ... {


  %init = memref.load %out[%row]
  %acc = scf.for %j ... {

    ...
  }
  memref.store %acc, %out[%row]

  gpu.terminator
}
```

- The store is Part 1's, indexed by `row`: after the reduction loop, inside the launch body.

Reference: `MatVecToGPUPattern`, Step 4 of 5 — Finish the Body

the C++

```
LogicalResult matchAndRewrite(MatVecOp op, ...) const override {
  // step 1: operands, host registration, c0, c1, numRows, numCols
  auto launchOp = gpu::LaunchOp::create(..., numRows, c1, c1);
  {
    // step 2: the insertion point, row = bx * blockDim.x + tx
    Value init = memref::LoadOp::create(..., out, ValueRange{row});
    auto redLoop = scf::ForOp::create(..., ValueRange{init});
    {
      // step 3: the reduction body
    }
    memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                            ValueRange{row});
    gpu::TerminatorOp::create(rewriter, loc);
  }
  rewriter.eraseOp(op);
  return success();
}
```

the IR it generates

```
gpu.launch ... {


  %init = memref.load %out[%row]
  %acc = scf.for %j ... {

    ...
  }
  memref.store %acc, %out[%row]

  gpu.terminator
}
```

- The store is Part 1's, indexed by `row`: after the reduction loop, inside the launch body.
- A launch body must end in `gpu.terminator`.

Reference: `MatVecToGPUPattern`, Step 5 of 5 — Registration

`LowerBlasToGPU.cpp`, the GPU pass:

```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToGPUPattern>(&getContext());
  // TODO(exercise): register the matmul pattern too:
  //   patterns.add<MatMulToGPUPattern>(&getContext());
  ...
}
```

Reference: `MatVecToGPUPattern`, Step 5 of 5 — Registration

`LowerBlasToGPU.cpp`, the GPU pass:

```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToGPUPattern>(&getContext());
  // TODO(exercise): register the matmul pattern too:
  //   patterns.add<MatMulToGPUPattern>(&getContext());
  ...
}
```

- Registered exactly as in Part 1. `MatMulToGPUPattern` goes on the same line.

Exercise 4 — Writing the GPU Lowering for Matmul

`MatMulToGPUPattern` in `blas-dialect/lib/LowerBlasToGPU.cpp`

- **Provided**, commented out: the operands, the host registration, `c0`, `c1`, `M`, `K`, `N`, and the `eraseOp`. Uncomment it.
- **Write** the `gpu::LaunchOp::create` and its body — the load, the reduction loop, the store, the `gpu::TerminatorOp` — then register the pattern.

What is different from matvec:

- A 2D thread grid: `N` threads in x (the columns of `out`), `M` in y (its rows) — the grid from step 1.
- Two global indices: `row` from the y ids and `col` from the x ids, each built the way matvec builds its `row`.
- The reduction runs over `K`, reading `a[row, k]` and `b[k, col]`, into `out[row, col]`.

10 minutes to write it — then we walk through the solution.

Solution — `MatMulToGPUPattern`

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, N, M, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  // by, bx, blockDimY, blockDimX, ty, tx: as matvec gets bx, blockDimX, tx
  Value rowBase = arith::MulIOp::create(rewriter, loc, by, blockDimY);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, ty);
  Value colBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value col = arith::AddIOp::create(rewriter, loc, colBase, tx);

  Value init =
      memref::LoadOp::create(rewriter, loc, out, ValueRange{row, col});
  auto redLoop =
      scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
  {
    // the guard, the insertion point, k and acc: as in matvec
    Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{row, k});
    Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, col});
    // prod, sum and the yield: as in matvec
  }
  memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                          ValueRange{row, col});
  gpu::TerminatorOp::create(rewriter, loc);
}
```

Solution — `MatMulToGPUPattern`

```
auto launchOp =          ← a 2D grid:  N threads in x, M in y
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, N, M, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  // by, bx, blockDimY, blockDimX, ty, tx: as matvec gets bx, blockDimX, tx
  Value rowBase = arith::MulIOp::create(rewriter, loc, by, blockDimY);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, ty);
  Value colBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value col = arith::AddIOp::create(rewriter, loc, colBase, tx);

  Value init =
      memref::LoadOp::create(rewriter, loc, out, ValueRange{row, col});
  auto redLoop =
      scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
  {
    // the guard, the insertion point, k and acc: as in matvec
    Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{row, k});
    Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, col});
    // prod, sum and the yield: as in matvec
  }
  memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                          ValueRange{row, col});
  gpu::TerminatorOp::create(rewriter, loc);
}
```

Solution — `MatMulToGPUPattern`

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, N, M, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  // by, bx, blockDimY, blockDimX, ty, tx: as matvec gets bx, blockDimX, tx
  Value rowBase = arith::MulIOp::create(rewriter, loc, by, blockDimY);    ← row from y
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, ty);
  Value colBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);  ← col from x
  Value col = arith::AddIOp::create(rewriter, loc, colBase, tx);

  Value init =
      memref::LoadOp::create(rewriter, loc, out, ValueRange{row, col});
  auto redLoop =
      scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
  {
    // the guard, the insertion point, k and acc: as in matvec
    Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{row, k});
    Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, col});
    // prod, sum and the yield: as in matvec
  }
  memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                          ValueRange{row, col});
  gpu::TerminatorOp::create(rewriter, loc);
}
```

Solution — `MatMulToGPUPattern`

```
auto launchOp =
    gpu::LaunchOp::create(rewriter, loc, c1, c1, c1, N, M, c1);
{
  OpBuilder::InsertionGuard guard(rewriter);
  rewriter.setInsertionPointToStart(&launchOp.getBody().front());
  // by, bx, blockDimY, blockDimX, ty, tx: as matvec gets bx, blockDimX, tx
  Value rowBase = arith::MulIOp::create(rewriter, loc, by, blockDimY);
  Value row = arith::AddIOp::create(rewriter, loc, rowBase, ty);
  Value colBase = arith::MulIOp::create(rewriter, loc, bx, blockDimX);
  Value col = arith::AddIOp::create(rewriter, loc, colBase, tx);

  Value init =
      memref::LoadOp::create(rewriter, loc, out, ValueRange{row, col});
  auto redLoop =
      scf::ForOp::create(rewriter, loc, c0, K, c1, ValueRange{init});
  {
    // the guard, the insertion point, k and acc: as in matvec
    Value aElem = memref::LoadOp::create(rewriter, loc, a, ValueRange{row, k});
    Value bElem = memref::LoadOp::create(rewriter, loc, b, ValueRange{k, col});
    // prod, sum and the yield: as in matvec
  }
  memref::StoreOp::create(rewriter, loc, redLoop.getResult(0), out,
                          ValueRange{row, col});     ← two indices
  gpu::TerminatorOp::create(rewriter, loc);
}
```

Solution — Register the Pattern

```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToGPUPattern, MatMulToGPUPattern>(&getContext());
  ...
}
```

Solution — Register the Pattern

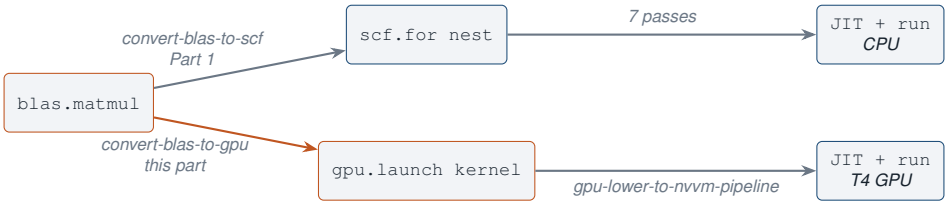
```
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  patterns.add<MatVecToGPUPattern, MatMulToGPUPattern>(&getContext());
  ...
}
```

- Forget it and the pass stops with:

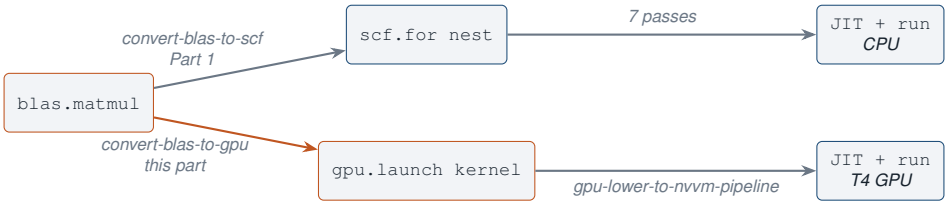
```
error:  convert-blas-to-gpu did not convert 'blas.matmul' -- is its pattern
written and registered in blas-dialect/lib/LowerBlasToGPU.cpp?
```

▶ Colab 2.28–2.32

Recap

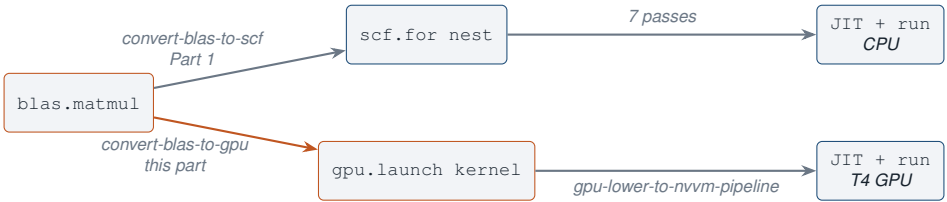

blas.matmul
convert-blas-to-scf
Part 1
scf.for nest
7 passes
JIT + run
CPU
convert-blas-to-gpu
this part
gpu.launch kernel
gpu-lower-to-nvvm-pipeline
JIT + run
T4 GPU

Recap



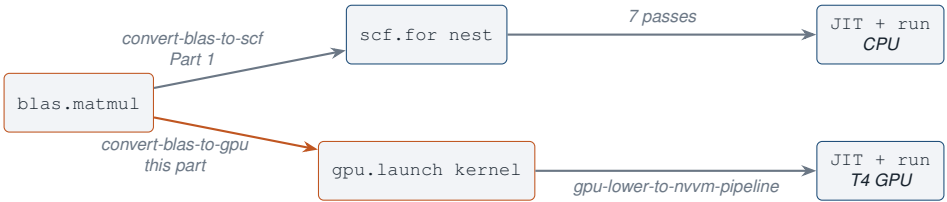
- One op definition, two lowerings: loops on the CPU in Part 1, a kernel on the GPU in this part.

Recap



- One op definition, two lowerings: loops on the CPU in Part 1, a kernel on the GPU in this part.
- The pattern decides how the work maps onto the GPU: one thread per output element, x on the columns.

Recap



- One op definition, two lowerings: loops on the CPU in Part 1, a kernel on the GPU in this part.
- The pattern decides how the work maps onto the GPU: one thread per output element, x on the columns.
- It builds the kernel read in step 1, and that runs through the same GPU pipeline.